

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 1 Solutions / Grading Guidelines

(1) Write down an **algorithm** for cooking your favorite recipe. (Don't cop out here – even if you don't know how to cook, you know how to make something to eat.) Assume the reader does not know how to cook.

Obviously there's no "right answer" here. The main thing we're looking for is sufficient specificity – did they write their process in an unambiguous fashion? Your judgment as to how well they accomplished that and how much partial credit to assign.

(2) Give an **algorithm** for computing $\lfloor \sqrt{n} \rfloor$. (Recall that $\lfloor \cdot \rfloor$ is the *floor function* – it is equal to the largest integer less than or equal to its input.) However, you may only use these operations: basic arithmetic (+, -, *, /), comparison (==, <, >, <=, >=), and assignment (=). (Note you do not necessarily have to use ALL of them.)

Again, lots of ways to do this. Key item, though, is specifying WHAT they will allow as legitimate input. 4 points off if they don't do this.

Here is a simple approach that assumes n is a non-negative real number:

- 1) Let $i = 1$
- 2) Compute $s = i * i$.
- 3) If $s > n$, return $i-1$.
- 4) Otherwise, $i = i + 1$ and goto 2.

(3) (a) Execute Euclid's algorithm by hand (you may use a calculator for the arithmetic) to find gcd (31415, 14142). Show all steps.

$$31415 \bmod 14142 = 3131$$

$$14142 \bmod 3131 = 1618$$

$$3131 \bmod 1618 = 1513$$

$$1618 \bmod 1513 = 105$$

$$1513 \bmod 105 = 43$$

$$105 \bmod 43 = 19$$

$$43 \bmod 19 = 5$$

$$19 \bmod 5 = 4$$

$$5 \bmod 4 = 1$$

$$4 \bmod 1 = 0 \rightarrow \text{The answer is 1.}$$

(b) Estimate how many times faster Euclid's algorithm will be than the CS1 algorithm on this particular pair of input values. Explain your answer.

Euclid's algorithm required 10 iterations; the CS1 algorithm will need 14142 (since we will count from 14142 all the way down to 1). If the loop iterations take about the same amount of time for both algorithms, then the CS1 algorithm will take > 1400 times as long.

(4) Prove that, for every pair of positive integers $m \geq n$, $\gcd(m, n) = \gcd(n, m \% n)$.

Lemma: For any integer a , if $a \% d = 0$, then for any integer $k \neq 0$, $ka \% d = 0$.

Proof: $a \% d = 0$ means $a = cd$ for some integer c . ka is thus $(kc)d$, which means ka is an integer multiple of d .

Lemma: For integers $a, b \geq 1$, if $a \% d = 0$ and $b \% d = 0$, then $(a \pm b) \% d = 0$.

Proof: $a = c_1d$ and $b = c_2d$, for integers c_1 and c_2 . $(a \pm b) = c_1d \pm c_2d = (c_1 \pm c_2)d$, which means $(a \pm b)$ is an integer multiple of d .

Claim: $\gcd(m, n) = \gcd(n, m \% n)$

Proof: Let $r = (m \% n)$, $d = \gcd(m, n)$, and $d' = \gcd(n, r)$.

Then $m \% d = 0$ and $n \% d = 0$. By the two lemmas above, $(m - kn) \% d = 0$ for any integer k . Observe that $r = m - kn$ for some nonnegative integer k . Therefore, $r \% d = 0$, and d is a common divisor of n and r . Thus, $d \leq d'$, because d' is the greatest of these.

Similarly, $(r + kn) \% d' = 0$. We know $m = r + kn$ (using the same k as above), and so $m \% d' = 0$. Thus, d' is a common divisor of m and n , and then $d' \leq d$.

Since $d \leq d'$, and $d' \leq d$, it must be that $d = d'$. ■

(5) Consider the standard **algorithm** for converting a positive integer from decimal to binary.

(a) Describe this algorithm in English.

(b) Describe this algorithm in pseudocode.

(a) Start with an empty bitstring. Repeatedly integer divide the number by two, appending the remainder each time to the left side of the bitstring. Stop when the quotient is zero.

(b)

```
def decToBin(n):      # n must be positive integer
    result = ""
    while n > 0:
        result = str(n % 2) + result
        n = n // 2     # integer division
    return result
```

(6) Given the lengths a, b, c of the three sides of a triangle, which of the following formulas for calculating the triangle's area would be considered a valid **algorithm**? (Note: All of the formulas are correct.)

(a) $A = \sqrt{s(s-a)(s-b)(s-c)}$ where $s = (a + b + c) / 2$

(b) $A = \frac{1}{2}ab \sin C$ where C is the interior angle between sides a and b

(c) $A = \frac{1}{2}bh_b$ where h_b is the length of the altitude of the triangle drawn to side b

Only the first of these is a valid algorithm, because all of the values needed are already specified. For (b), there is no indication how to find the angle between sides a and b . For (c), there is no indication how to find the length of the altitude.

(7) Consider the problem of calculating n^k , for positive integers n and k .

(a) Like GCD, this is a standard CS1 task involving a simple loop. Write pseudocode for an **algorithm** for determining this value that performs approximately k multiplications.

(b) For large values of k , this is fairly terrible. There is a method for finding n^k that requires only around $\lg k$ multiplications. (Note: Throughout the course, I will use $\lg \cdot$ to indicate the base-2 logarithm.) Write pseudocode for such an **algorithm**.

```
def bad_power(n,k):    # both inputs must be positive integers
    result = 1
    for i = 1 to k do:
        result = result * n
    return result
```

```
def better_power(n,k):    # both inputs must be positive integers
    result = 1
    Let s = decToBin(k)    # Turn k into a bitstring!
    for i = 0 to len(s)-1: # Walk down the bitstring left to right
        result = result * result
        if s[i] == '1':
            result = result * n
    return result
```

Note I'm not expecting anyone to actually convert into a bit string – anything that touches the repeated squaring idea should get full credit. (Honestly, this one should have been a “describe in English” instead of a pseudocode description.)